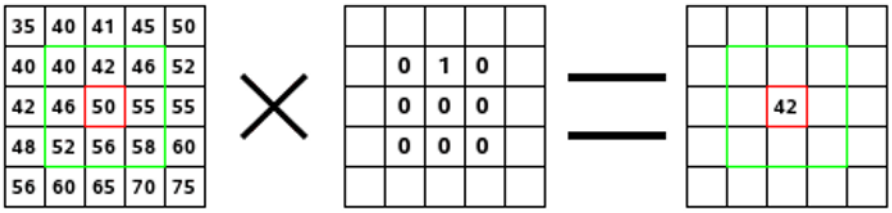
Laboratory 1: Applying a Convolution Matrix

Working time: 2 weeks (deadline – week 4)

Many image filters use the convolution operation based on a convolution matrix. Simply put, convolution is the treatment applied to a matrix F of size n x m (which can represent an image represented by pixels) through another matrix called the convolution matrix - C.
Most of the time, convolution matrices of size 3x3 or 5x5 are used. Generalizing, we can consider a convolution matrix of size kxk where k is an odd number.
By applying convolution, each element of the initially given matrix is modified.

First we can consider the case where the convolution result is stored in a new matrix V of the same dimensions as the initial matrix F (nxm).

Example for a 3x3 convolution matrix C (k=3) applied to a given 5x5 matrix F (m=5, n=5)



The calculation performed for the element marked in red with the value 50 is as follows:
(40*0)+(42*1)+(46*0) + (46*0)+(50*0)+(55*0) + (52*0)+(56*0)+(58*0) = 42

Generalizing and considering that a new matrix results from applying convolution, which we denote by V, we can express the calculation performed with a 3x3 convolution matrix as follows:
v[i,j] =
f[i,j] * c[1,1] +
f[i-1,j] * c[0,1] +
f[i,j-1] * c[1,0] +
f[i-1,j-1] * c[0,0] +
f[i+1,j] * c[2,1] +
f[i,j+1] * c[1,2] +
f[i+1,j+1] * c[2,2]
where 0 <= i < n and 0 <= j < m.

It is considered that the initial matrix is virtually bordered with elements equal to the boundary elements. Thus: f[-1,j]=f[0,j], f[n,j]=f[n-1,j], f[i,-1]=f[i,0], f[i,m]=f[i,m-1], f[-1,-1]=f[0,0], f[-1,m]=f[0,m-1], f[n,-1]=f[n-1,0], f[n,m]=f[n-1,m-1].

ASSIGNMENT:

Given a matrix F(n,m) and a convolution matrix C(k,k), calculate the resulting matrix V(n,m) obtained by applying convolution with matrix C on matrix F.

• Sequential program
• Parallel program: use **p** threads for the calculation.
Objective: The most balanced and efficient distribution of calculations across threads!

For dividing computational tasks, **geometric decomposition** (data distribution) will be used with the following 2 variants:
1. **Horizontal** (multiple rows allocated to a thread)
2. **Vertical** (multiple columns allocated to a thread)

Extra points (2 points for each extra variant added to the Lab1 grade) for the following variants:
• **Block** – submatrices allocated to a thread (division is done both horizontally and vertically)
• Distribution based on a **delta distribution** function by which a thread with index t is assigned a subset of indices from the matrix:
delta : N x M -> P, N={0,1,2...n-1}, M={0,1,2...m-1}, P={0,1,2...p-1}
delta(i,j) = t, 0 <= i < n, 0 <= j < m
Testing will be done by defining the distribution function to obtain:
- **Linear distribution** (adjacent indices assigned to the same thread) or
- **Cyclic distribution** (with step equal to p).

Implementation Details:

• Input data is read from an input file named "MatrixF.txt ", "ConvolutionM.txt"
• The files must be created beforehand by adding randomly generated numbers.
• All runs (10 executions) must be executed using the same input files.

• Verification of the output file against the output file obtain for the sequential variant should be done for each of the 10 executions!
• Output data: "output.txt" – a file containing the result matrix.
• Implementation languages:
- Java
- C++ (at least C++11)
  * matrices allocated statically (int f[MAX][MAX])
  * matrices allocated dynamically (new...)
• **Direct use of threads (explicit creation) => Executors are NOT allowed.**

Testing:

Measure the execution time only for the computation! Do not include in the execution time the time needed for reading and writing.

Test cases:
• n = m = 10 and k = 3; p = 4
• n = m= 1000 and k = 3; p = 2, 4, 8, 16
• n = 10, m = 10000 and k = 5; p = 2, 4, 8, 16
• n = 10000, m = 10 and k = 5; p = 2, 4, 8, 16
• n = 10000, m = 10000 and k = 5; p = 2, 4, 8, 16

Performance Documentation:

For each of the test variants resulting from different implementations and different test data, analyze performance using tables similar to the ones shown below. These tables must be added to the documentation!

Java:

Matrix Type	Nr threads	Execution Time	Speedup (Tseq/Tpar)

C++

Matrix Type	Allocation Type	Nr threads	Execution Time	Speedup (Tseq/Tpar)

Observations:

• Each test must be repeated 10 times, and the arithmetic mean of the 10 runs is considered for evaluation.
• For each variant (sequential, parallel), use the same input files.
• Follow the recommendations in the "Testing" and "Correctness Verification" files.

Analysis:

• Compare performance for each case – sequential versus parallel and parallel variants against each other.
• Compare execution times obtained with the Java implementation versus the C++ implementation.
• Compare the two allocation variants for the C++ implementation.
• The analysis must be highlighted in the documentation.

For each matrix size extract the best variant from the performance point of view.